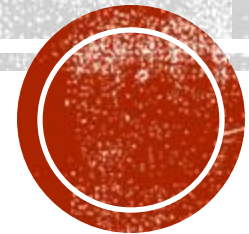


API

Interface de Programação de Aplicativos



O QUE É UMA API?

- “... é um conjunto de regras ou protocolos que permitem que aplicativos de software se comuniquem entre si para trocar dados, recursos e funcionalidades.”



O QUE É UMA API?

- “... é um conjunto de regras ou protocolos que permitem que aplicativos de software se comuniquem entre si para trocar dados, recursos e funcionalidades.”
- Segundo a IBM, APIs são contratos que definem como sistemas trocam dados, impulsionando tudo, desde apps bancários até redes sociais.”



DEFINIÇÃO TÉCNICA

- Acrônimo para Application Programming Interface.
- Conjunto de regras que permite que softwares se comuniquem.
- Funciona como um "contrato": se o sistema A envia X no formato Y, o sistema B responde com Z.



DEFINIÇÃO TÉCNICA

- Tecnicamente, uma API é um intermediário padronizado. Por exemplo: quando você usa um app de clima, ele acessa dados de um servidor via API. Seu app **não precisa saber como os dados são coletados**, apenas como solicitá-los.
- A IBM destaca que APIs **abstraem complexidade**, acelerando o desenvolvimento.



A ANATOMIA DE UMA SOLICITAÇÃO API

1. Cliente envia uma requisição (ex: "listar produtos").
2. API valida e traduz a requisição.
3. Servidor processa e envia dados à API.
4. API formata a resposta (ex: JSON) e entrega ao cliente.



A ANATOMIA DE UMA SOLICITAÇÃO API

1. Desmontando o processo:
2. A aplicação faz um pedido à API como 'GET /produtos'.
3. A API **roteia esse pedido ao servidor correto**, que retorna os dados.
4. A API então os **organiza em JSON** (formato leve para dados) e os entrega.
5. Tudo em milissegundos!



TIPOS DE APIS

- **Públicas** (Open APIs): Acessíveis a todos (ex: Google Maps, X(Twitter)).
- **Privadas** (Internal APIs): Uso exclusivo dentro de uma empresa.
- **De Parceiros**: Integração controlada entre empresas (ex: sistemas de pagamento).



BENEFÍCIOS-CHAVE:

- Inovação mais rápida: Reutilize funcionalidades (ex: login com Facebook).
- Escalabilidade: Serviços independentes crescem sem travar o sistema.
- Experiência do usuário: Dados integrados = apps mais ricos (ex: Uber usando Maps).
- Monetização: Empresas vendem acesso a APIs (ex: API de pagamentos).



BENEFÍCIOS-CHAVE:

- APIs são **aceleradoras de negócios**.
- A IBM ressalta que elas permitem **focar na sua especialidade** enquanto terceiriza o resto.
- Exemplo: um e-commerce usa API de correios para fretes e de pagamento para transações. Isso **reduz custos e libera tempo!**



SEGURANÇA EM APIS

- Chaves de API: Identificam quem acessa.
- Tokens OAuth: Permissões temporárias (ex: "app X pode ler e-mails, mas não enviar").
- Criptografia: Dados trafegam via HTTPS.
- Gateways: Firewalls para APIs (controlam tráfego e bloqueiam ameaças).



CASOS REAIS

- Bancos: APIs para pix, saldo e investimentos.
- Saúde: Integração entre prontuários e laboratórios.
- IoT: Sensores enviam dados para nuvem via API.

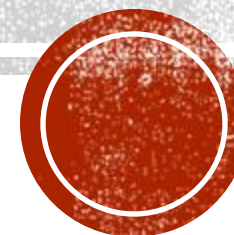


CONCLUSÃO

- Resumo: APIs são intermediárias essenciais para integração.
- Futuro: Tendência em APIs inteligentes (com IA) e GraphQL (consultas personalizadas).
- APIs são a **infraestrutura invisível que conecta nosso mundo**. Elas democratizam tecnologia, aceleram negócios e criam ecossistemas. Como diz a IBM: **'Sem APIs, não há transformação digital'**



REST



REST

- Acrônimo para **REpresentational State Transfer**.
- **Estilo arquitetural** (não protocolo!) criado por Roy Fielding em 2000.
- Define **regras para comunicação entre sistemas** via HTTP.



06 PRINCÍPIOS

- **Cliente-Servidor:** Separação clara de responsabilidades.
- **Stateless:** Cada requisição contém **toda informação necessária** (sem sessões).
- **Cacheable:** Respostas devem ser declaradas como *cacheáveis* ou não.
- **Interface Uniforme:** URIs consistentes + métodos HTTP padronizados.
- **Sistema em Camadas:** Intermediários (proxies, gateways) são transparentes.
- **Código Sob Demanda** (opcional): Execução de scripts no cliente.



CLIENTE-SERVIDOR

- Separação do cliente e do armazenamento de dados (servidor), dessa forma, poderemos ter uma portabilidade do sistema, usando o React para WEB e React Native para o smartphone, por exemplo.



STATELESS

- Cada requisição que o cliente faz para o servidor, deverá conter todas as informações necessárias para o servidor entender e responder (RESPONSE) a requisição (REQUEST).
- Exemplo: A sessão do usuário deverá ser enviada em todas as requisições, para saber se aquele usuário está autenticado e apto a usar os serviços, e o servidor não pode lembrar que o cliente foi autenticado na requisição anterior.



CACHEABLE

- As respostas para uma requisição, deverão ser explicitas ao dizer se aquela requisição, pode ou não ser cacheada pelo cliente.
- Quando possível, os recursos devem ser armazenados em cache no lado do cliente ou do servidor. As respostas do servidor também precisam conter informações sobre se o cache é permitido para o recurso entregue. O objetivo é melhorar o desempenho no lado do cliente e, ao mesmo tempo, aumentar a escalabilidade no lado do servidor.



INTERFACE UNIFORME

- Manter uma uniformidade, uma constância, um padrão na construção da interface.
- A API precisa ser coerente para quem vai consumi-la. Precisa fazer sentido para o cliente e não ser confusa. Logo, coisas como: o uso correto dos verbos HTTP; endpoints coerentes (todos os endpoints no plural, por exemplo); usar somente uma linguagem de comunicação (json) e não várias ao mesmo tempo; sempre enviar respostas aos clientes; são exemplos de aplicação de uma interface uniforme.



SISTEMA EM CAMADAS

- O cliente acessa a um endpoint, sem precisar saber da complexidade, de quais passos estão sendo necessários para o servidor responder a requisição, ou quais outras camadas o servidor estará lidando, para que a requisição seja respondida.
- Nas APIs REST, as chamadas e respostas passam por diferentes camadas.
- Como regra geral, não assuma que as aplicações do cliente e do servidor se conectam diretamente uma à outra.
- Pode haver vários intermediários diferentes no loop de comunicação.
- As APIs REST precisam ser projetadas de modo que nem o cliente nem o servidor possam dizer se estão se comunicando com a aplicação final ou um intermediário.

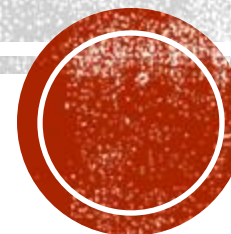


CÓDIGO SOB DEMANDA

- As APIs REST geralmente enviam recursos estáticos, mas, em certos casos, as respostas também podem conter código executável (como miniaplicativos Java). Nesses casos, o código só deve ser executado sob demanda.



RESTFUL



RESTFUL...

- ...é a aplicação de todos os padrões REST.



REFERÊNCIA

- <https://www.ibm.com/br-pt/think/topics/api>
- <https://www.ibm.com/br-pt/think/topics/rest-apis>



EXERCÍCIO

- Pesquisar e implementar + 2 APIs gratuitas em uma linguagem de sua preferência.
- Entregar junto com a do Gemini.

